

SETU — SYSTEM ARCHITECTURE SPECIFICATION

5-Layer Engineering Pipeline: On-Device Edge Sensing, Server Clustering, Multi-Stage Verification, Municipal GIS Portal & Active Feedback Loop

LAYER 1: MOBILE SDK (On-Device — The Edge)

MEASUREMENTS (Sampled Continuously):

• 3-Axis Accelerometer (x, y, z) — 100 Hz	• GPS Location (lat, lon) — 1 Hz / Passive (host app)
• 3-Axis Gyroscope (x, y, z) — 100 Hz	• GPS Accuracy (metres) — 1 Hz
• Gravity Vector (x, y, z) — 100 Hz / Continuous	• Vehicle Speed (km/h) — 1 Hz
• Timestamp (epoch milliseconds) — Per-event	• Vehicle Heading (degrees) — 1 Hz

CALCULATIONS & SIGNAL CLEANING (On-Device DSP):

- Linear Resampling — Normalize jittery Android sensor ticks to fixed 100 Hz grid
- Dynamic Acceleration — Subtract gravity from raw accelerometer
- Vehicle-Frame Rotation — Use gravity vector + heading to build rotation matrix, convert phone axes → vehicle vertical / longitudinal / lateral
- Total Shock Magnitude — $\sqrt{aX^2 + aY^2 + aZ^2}$ — rotation-invariant fallback
- Band-Pass Filter — Butterworth 0.5–30 Hz, isolate pothole energy band
- Per-Device Baseline — Rolling mean (μ) and std dev (σ) over smoothest 20% of driving
- Normalized Shock — Z-score: $(a_vert - \mu) / \sigma$
- Peak Vertical Impact — Maximum $|a_vertical|$ in 2-second window
- Maximum Jerk — $\max(|da/dt|)$ — rate of change of vertical acceleration
- Windowed RMS — $\sqrt{(1/N \times \sum z_i^2)}$ — sustained roughness measure
- Event Duration — Consecutive samples above threshold $\times 10ms$
- Cross-Axis Correlation — $Corr(a_vertical, a_lateral)$ — pothole roll vs bump pitch
- Threshold Gate — if $|z_norm| < 3.0\sigma$ AND $RMS < r_0 \rightarrow DROP$ (saves battery, kills ~99% of windows)
- Local AI Classification — 1D-CNN (~120 KB, INT8, LiteRT) classifies surviving windows into 6 classes: smooth_road / rough_road / POTHOLE / speed_bump / rumble_strip / non_road_event
- Local AI Confidence Score — Softmax probability for the predicted class
- Road Segment Pass Count — Silent passes over each road segment (the denominator)

INFORMATION SENT TO SERVER (Batched every 60s, gzipped JSON, ~200 bytes/event):

1. Hashed Device ID: SHA-256(install_id + salt), no PII	9. Travel Direction: Heading (0–360°)
2. Vehicle Type: two_wheeler / car / auto / bus	10. Defect Class: pothole / rough_road
3. Phone Category: budget / midrange / flagship	11. Local AI Conf.: 0.0 to 1.0
4. Device Baseline: μ_dev , σ_dev (noise floor)	12. Vertical Force: Peak vertical acc (m/s^2)
5. Timestamp: Epoch milliseconds	13. Impact Jerk: Max jerk (m/s^3)
6. Location: (lat, lon) of defect	14. RMS Windowed: Roughness score
7. GPS Accuracy: 68% conf. radius (m)	15. Event Duration: Impact duration (ms)
8. Vehicle Speed: km/h at moment of event	16. Pass Counts: (segment_id, count) pairs

TECH USED IN THIS LAYER:

- Kotlin + Android SDK (native) (or Flutter fallback) • SensorManager 100 Hz • TYPE_GRAVITY orientation
- FusedLocationProviderClient GPS • LiteRT (TFLite) on-device AI • Room SQLite offline queue • WorkManager upload

LAYER 2: BACKEND SERVER (Cloud — The Brain) — Part 1: Ingestion & Clustering

DATA RECEIVED: All 16 telemetry fields from Layer 1 (batched from thousands of fleet and commuter devices).

SERVER-SIDE PROCESSING (Stages A to D):

Stage A — Ingestion & Validation

- Payload Schema Validation: Pydantic v2 rejects malformed or out-of-range data before touching the database.
- Rate Limiting: slowapi throttles excessive submissions per device hash to prevent spam or runaway loops.
- Speed Gate: Discard events where speed < 5 km/h (walking/handling artifacts) or > 150 km/h (erroneous spikes).
- GPS Accuracy Gate: Discard events where GPS accuracy radius > 30m.
- Timestamp Sanity: Reject future timestamps or events older than 48 hours.
- Duplicate Suppression: Same device hash + same location (within 5m) + within 60 seconds → retain highest-confidence event only.

Stage B — Normalization & Trust Scoring

- Speed-Normalized Impact: $\text{impact_norm} = \text{vertical_impact} / \sqrt{\text{speed}}$ — equalizes impact severity readings across varying vehicle speeds.
- Device Trust Score: $\text{weight} = f(\text{phone_category}, \text{gps_accuracy}, \text{baseline_noise})$ — flagship phones with tight GPS and low noise floor receive highest initial weighting.
- Device Reputation: Trust score increases when device reports align with confirmed defect clusters; drops when reports consistently land in rejected noise clusters.

Stage C — Geo-Processing

- Map Matching: Snap raw GPS coordinates to nearest road centreline using self-hosted OSRM with OpenStreetMap data. Eliminates perpendicular GPS drift (~27–32m in dense urban corridors).
- Road Segment Assignment: Tag each event with its corresponding OSM Way ID and segment identifier.
- Lane Determination: Utilizes vehicle heading combined with road geometry to determine precise travel direction on divided dual-carriageways.
- Ward / Zone Lookup: Spatial join against municipal boundary polygons to tag the administrative ward responsible for maintenance.
- H3 Hexagonal Indexing: PostGIS spatial indexing for scalable multi-resolution spatial aggregation.

Stage D — DBSCAN Spatial Clustering

- Clustering: PostGIS ST_ClusterDBSCAN(geom , $\text{eps} := 20$, $\text{minpoints} := 5$) on map-matched points. Groups events within 20m having ≥ 5 reports.
- Distinct Device Requirement: Cluster requires reports from ≥ 5 DISTINCT device hashes. 40 reports from 1 device $\approx$ 1 report, not 40. Per-device contribution capped at 3.
- Inverse-Variance Weighted Centroid: Confirmed location = $\Sigma(w_i \times \text{loc}_i) / \Sigma(w_i)$ where $w_i = 1/\text{accuracy}^2$ — tighter GPS readings dominate the centroid calculation.
- Confidence Ellipse: Uncertainty reported as an error ellipse, displayed on the dashboard as a confidence-scaled circle.
- Cluster Confidence Score: $\text{cluster_conf} = 1 - \prod(1 - \text{conf}_i)$ for all reports (e.g., 5 distinct devices at 85% confidence yield 99.99% aggregate cluster confidence).
- Vehicle Diversity Bonus: Clusters containing both two-wheelers and four-wheelers receive higher confidence scores, ruling out vehicle-specific suspension harmonics.

LAYER 2: BACKEND SERVER (Cloud — The Brain) — Part 2: Analytics & Infrastructure

SERVER-SIDE PROCESSING (Stages E to G):

Stage E — Defect Analysis

- Classification Consensus: Majority vote across reports in cluster (e.g., 8 say 'pothole', 2 say 'rough_road' → confirmed POTHOLE).
- Severity Score: $\text{severity} = (0.4 \times \text{avg_impact_norm}) + (0.3 \times \text{avg_jerk}) + (0.2 \times \text{max_impact}) + (0.1 \times \text{avg_duration})$.
Categorized into: LOW / MEDIUM / HIGH / CRITICAL.
- Defect Size Estimation: $\text{width_m} \approx \text{avg_speed_m_s} \times \text{avg_duration_s}$.
- Detection Rate: $\text{pothole_detections} / \text{total_passes_on_segment}$. ($50/500 = 10\% = \text{real defect}$; $3/10,000 = 0.03\% = \text{background noise}$).
- Road Roughness Index: Aggregates RMS roughness values per road segment across all devices over 30 days to build a continuous roughness heatmap.

Stage F — Temporal Intelligence

- New vs Known: Checks if a confirmed defect already exists within 20m. If yes → updates existing record (attaches new telemetry evidence); if no → instantiates new cluster.
- Trend Analysis: Tracks severity progression over weeks (growing defect vs stabilized vs sudden drop to 0 indicating road resurfacing).
- Repair Verification ('Verified by Silence'): When municipal contractors mark a pothole as 'repaired', SETU monitors passive telemetry. Zero detections from 100+ subsequent passes over 7 days → repair confirmed ✓. If detections resume → repair failed, ticket auto-reopened ✗.
- Seasonal Adjustment: During monsoon months (June–September): automatically lowers minpoints threshold to accommodate rapid pothole formation and reduced traffic volumes.

Stage G — Infrastructure Learning

- Speed Bump Whitelisting: Consistent detections at same location with low jerk + long duration + symmetric pitch → auto-tagged as speed bump. Future triggers auto-suppressed.
- Railway Crossing / Expansion Joint Mapping: Periodic multi-spike patterns at fixed locations are mapped as permanent roadway infrastructure.
- Suppression Zones: Persistent false-positive areas (steep driveways, parking ramps) receive down-weighting ($\times 0.2$) with escalating TTL: 3 days → 7 days → 30 days.

DATA STORED (PostgreSQL + PostGIS):

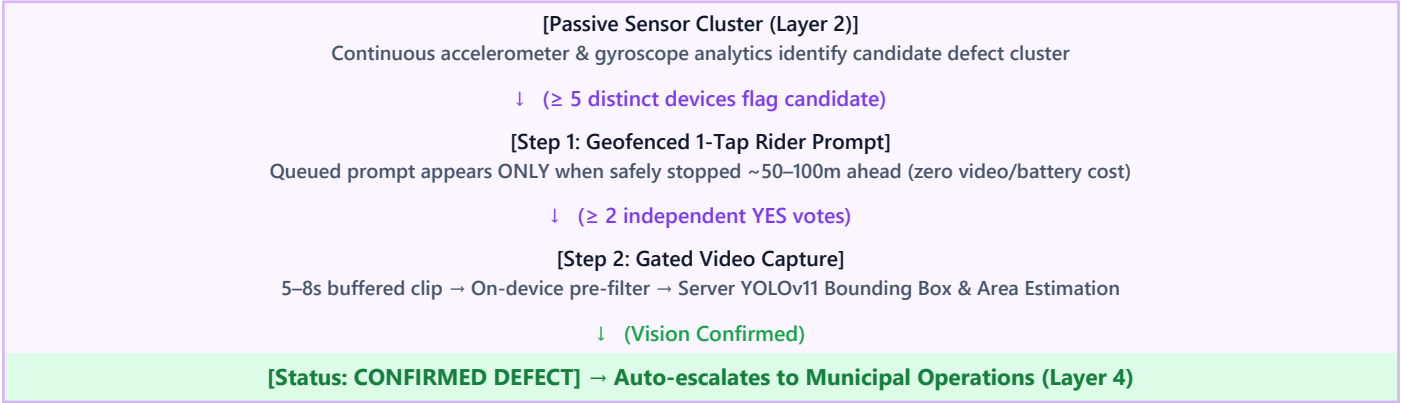
- Raw events table — All incoming reports with full DSP and orientation metadata
- Clusters table — Confirmed defects with centroid, confidence ellipse, severity, status, and evidence count
- Pass counts table — Per-segment, per-device silent pass tracking (the denominator for statistical confidence)
- Infrastructure whitelist — Known speed bumps, railway crossings, bridge expansion joints
- Device reputation table — Rolling trust scores per device hash
- Road segments table — Roughness index, aggregate RMS, detection rate per OSM segment
- Work orders table — Municipal repair tickets with status, SLA clock, contractor assignment, and verification logs

TECH USED IN THIS LAYER:

- Python 3.11+ / FastAPI — Async API server • Pydantic v2 — Strict schema validation
- PostgreSQL 16 + PostGIS 3 — Spatial database, DBSCAN, GiST spatial indexing • Redis 7 — Broker & hot cache
- Celery + Celery Beat — Background task workers & 5-minute clustering schedule • OSRM (Docker) — Road map matching
- scikit-learn — DBSCAN, Random Forest baseline • SQLAlchemy 2.0 + Alembic — ORM & migrations
- MinIO (dev) / S3 (prod) — Video storage with pre-signed URLs • Docker Compose • Deploy: Railway / Render / Fly.io

LAYER 3: MULTI-STAGE VERIFICATION (Ground-Truth Confirmation)

The system enforces a Cost-Gated Escalation Ladder so that expensive resources (camera operation, video upload bandwidth, municipal contractor dispatch) are ONLY triggered when lower-cost statistical filters have already confirmed a candidate cluster.



Step 1: Geofenced 1-Tap Human Verification (The Cheap Gate)

- When it triggers: Backend identifies an unverified DBSCAN candidate cluster (≥ 5 distinct device pings).
- Where it triggers: Backend arms a 50–100m geofence around the candidate defect location.
- Safety Gate (Non-Negotiable): The app NEVER prompts a rider while moving. The prompt is queued and ONLY appears when the vehicle comes to a complete stop (speed < 3 km/h for > 5 s at a traffic signal or delivery drop-off):
"You recently passed a bumpy spot on MG Road — was that a pothole?"
[✓ YES] [✗ NO] [? NOT SURE]
- Limits: Capped at ≤ 2 prompts per rider per day to prevent notification fatigue.
- Outcome: Filters out 85–90% of accidental sensor triggers (rough patches, manhole covers) without consuming video bandwidth or battery.

Step 2: Gated Video Capture & Vision Confirmation (The Expensive Gate)

- Trigger Condition: ONLY fires after 2 independent riders vote YES on the 1-tap prompt (or distinct high-magnitude sensor spikes).
- Camera Trigger: Next opted-in rider approaching the location with a handlebar-mounted phone (detected via stable landscape gravity vector) receives a silent trigger to buffer a 5–8 second rear camera clip.
- On-Device Pre-Filter: Discards clip immediately if scene is underexposed, heavily blurred, or phone is detected inside a bag/pocket.
- Cloud Vision Inference: Uploaded clip is processed on the server by YOLOv11 → verifies pothole bounding box, calculates approximate surface area (m²), and generates a compressed visual evidence thumbnail for municipal engineers.
- Privacy & DPDP Compliance: Raw video is permanently purged after 7 days; only the anonymized, cropped defect bounding box is retained in the database.

LAYER 4: MUNICIPAL WEB PORTAL (The Dashboard)

DATA RECEIVED FROM BACKEND: Confirmed defects (coordinates, severity, classification, evidence count, unique devices, timestamps), road roughness heatmap data (per-segment RMS), ward-level aggregate metrics, work order status logs, live updates via WebSocket.

DASHBOARD SCREENS:

Screen 1 — Live GIS Map (The Main Screen)

- deck.gl Layers: ScatterplotLayer (potholes as GREEN → YELLOW → ORANGE → RED dots), HeatmapLayer (roughness overlay), and PathLayer (road segment health).
- Map Controls & Dynamic Fly-To: Google Maps / MapLibre basemap toggle. Filter by severity, ward, date, and repair status. Newly confirmed defects pulse dynamically and auto-center the viewport. Clicking a dot opens Defect Detail Drawer.

Screen 2 — Defect Detail (Slide-over Drawer)

- Evidence & Metrics: Video crop with bounding box, P1–P4 severity score, estimated surface area (m²), evidence summary (e.g. "47 reports from 31 vehicles • 6 rider confirmations • verified in 2 clips").
- Timeline & Action Dispatch: First Seen → Confirmed → Work Order Created → Assigned → Under Repair → Verified. One-click assignment to ward engineer or work order export.

Screen 3 — Ward Scorecard

- Comparative League Table: Ward | Open Defects | P1 Critical | Avg Resolution Time | Failed Repairs | Road Quality Score. Drives gamified municipal accountability with 90-day trend sparklines.

Screen 4 — Analytics & Reports

- Visualization Suite: Defect trends over time (line charts), repair SLA by ward (bar charts), top 10 worst roads (ranked table), monsoon vs dry season analysis, hourly detection heatmaps. One-click PDF export.

Screen 5 — Work Order Management

- Lifecycle Tracking: Created → Assigned → Under Repair → Repair Claimed → Verified ✓ / Failed ✗. Real-time SLA countdown turns red if overdue. Auto-escalates unassigned critical defects > 24 hrs.

TECH USED IN LAYER 4:

- React 18 + TypeScript + Vite • deck.gl (ScatterplotLayer, HeatmapLayer, PathLayer) • Google Maps JS / MapLibre GL
- Zustand state • TanStack Query caching • Tailwind CSS + shadcn/ui • Recharts • framer-motion • WebSocket sync

LAYER 5: MODEL IMPROVEMENT (Post-Launch Feedback Loop)

INITIAL MODEL BASELINE: Initial model trained on public datasets. Expected baseline: ~70–78% single-device accuracy → ~85–90% after spatial clustering.

WHAT THE FEEDBACK LOOP PRODUCES:

- Growing labelled dataset of REAL Indian road events — The proprietary SETU-IND-1 dataset.
- Vehicle-specific calibration profiles — Activa vibrates differently from Pulsar from Swift.
- Phone-specific noise profiles — Budget Redmi vs mid-range vs flagship Samsung/iPhone.
- Regional road pattern data — Mumbai coastal roads ≠ Lucknow asphalt ≠ Bangalore ring roads.
- Speed bump / infrastructure map — Auto-built, auto-expanding permanent feature catalog.
- Seasonal patterns — Monsoon = more potholes, distinct vibration signatures on waterlogged roads.

TECH USED IN LAYER 5:

- Label Studio (self-hosted) • MLflow / W&B experiment tracking • PyTorch (1D-CNN retraining)
- Ultralytics YOLO (vision model) • Google Colab / Kaggle T4 GPU • LiteRT converter (INT8) • Remote model push